📋 Cahier des Charges — fastdoc

> Outil CLI + GitHub Action pour la génération automatique de documentation technique via l'IA.

---

1. Présentation du projet

1.1 Contexte

Les développeurs passent un temps considérable à rédiger et maintenir la documentation technique de leurs projets. Cette documentation est souvent incomplète, obsolète ou inexistante. **fastdoc** automatise cette tâche en analysant le code source et en générant une documentation professionnelle via l'IA.

1.2 Objectifs

- Permettre à tout développeur de générer une documentation technique en **une seule commande**
- Intégrer la génération de doc dans les pipelines CI/CD via une **GitHub Action**
- Supporter les langages et frameworks les plus utilisés dans l'écosystème du projet
- Produire une documentation structurée, lisible et exportable

1.3 Public cible

- Développeurs individuels (projets open source, portfolios)
- Équipes de développement (intégration CI/CD)
- Entreprises souhaitant automatiser leur documentation interne

---

2. Périmètre fonctionnel

2.1 Langages et frameworks supportés (v1.0)

Langage / Framework	Extensions analysées
JavaScript	`.js`, `.mjs`, `.cjs`
TypeScript	`.ts`
PHP / Laravel	`.php`
Dart / Flutter	`.dart`

Langage / Framework	Extensions analysées
Python	`.py`
Node.js	`.js` (détection via `package.json`)
NestJS	`.ts` (détection via décorateurs NestJS)

2.2 Formats de sortie

Format	Description
Markdown (`.md`)	Documentation lisible, versionnable dans Git
JSON (`.json`)	Sortie structurée pour intégrations tierces (portails doc, dashboards)

2.3 Fonctionnalités principales

CLI (`fastdoc`)

Commande	Description
`fastdoc generate`	Analyse le projet courant et génère la documentation
`fastdoc generate --file <path>`	Génère la doc d'un seul fichier
`fastdoc generate --lang <lang>`	Force le langage (ex: `--lang php`)
`fastdoc generate --output <format>`	Choisit le format de sortie (`md`, `json`, `all`)
`fastdoc generate --out-dir <path>`	Dossier de sortie (défaut: `./docs`)
`fastdoc init`	Initialise un fichier de configuration `.fastdocrc`
`fastdoc --version`	Affiche la version installée
`fastdoc --help`	Affiche l'aide

GitHub Action

- Déclenchement automatique sur `push` ou `pull_request`
- Génération de la documentation et commit automatique dans le repo
- Support des secrets GitHub pour la clé API

2.4 Contenu de la documentation générée

Pour chaque fichier analysé, fastdoc génère :

- **Description générale** du fichier / module
- **Liste des fonctions / méthodes** avec :
  - Nom et signature
  - Description du rôle
  - Paramètres (nom, type, description)
  - Valeur de retour (type, description)
  - Exemples d'utilisation
- **Classes et interfaces** (propriétés, méthodes, héritage)
- **Routes API** (pour Laravel, NestJS, Express)
- **Dépendances et imports** notables
- **Avertissements** (fonctions dépréciées, TODO, FIXME détectés)

---

3. Architecture technique

3.1 Stack technologique

Composant	Technologie
Langage principal	TypeScript
Runtime	Node.js >= 18
Parser de code	Tree-sitter
IA	Anthropic Claude API (claude-sonnet)
CLI framework	Commander.js
Tests	Vitest
Linting	ESLint + Prettier
Distribution	npm (package public)
CI/CD	GitHub Actions

3.2 Structure du projet

```
fastdoc/
├── src/
│   ├── cli/            # Commandes CLI (Commander.js)
│   │   ├── index.ts
│   │   ├── generate.ts
│   │   └── init.ts
│   ├── parser/         # Analyse du code source (Tree-sitter)
│   │   ├── index.ts
│   │   ├── javascript.ts
│   │   ├── typescript.ts
│   │   ├── php.ts
│   │   ├── dart.ts
│   │   └── python.ts
│   ├── ai/             # Intégration Claude API
│   │   ├── client.ts
│   │   └── prompts.ts
│   ├── generators/     # Génération des fichiers de sortie
│   │   ├── markdown.ts
│   │   └── json.ts
│   ├── config/         # Gestion de la configuration
│   │   └── index.ts
│   └── utils/          # Utilitaires
│       ├── file.ts
│       └── logger.ts
├── action/             # GitHub Action
│   ├── action.yml
│   └── entrypoint.ts
├── tests/
│   ├── parser/
│   ├── generators/
│   └── fixtures/       # Fichiers de code pour les tests
├── docs/               # Documentation générée par fastdoc lui-même
├── .fastdocrc.example  # Exemple de configuration
├── package.json
├── tsconfig.json
├── .eslintrc.json
└── README.md
```

3.3 Flow de traitement

```
Fichier(s) source
    ↓
[Parser]        → Extraction AST (fonctions, classes, routes...)
    ↓
[Prompt Builder] → Construction du prompt pour Claude
    ↓
[Claude API]     → Génération de la documentation en langage naturel
    ↓
[Generator]      → Mise en forme (Markdown / JSON)
    ↓
Fichier(s) de documentation
```

---

4. Configuration

4.1 Fichier `.fastdocrc`

```json
{
  "language": "auto",
  "output": ["md", "json"],
  "outDir": "./docs",
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist", "*.test.ts"],
  "ai": {
    "model": "claude-sonnet-4-20250514",
    "language": "fr"
  }
}
```

4.2 Variables d'environnement

Variable	Description	Obligatoire
`ANTHROPIC_API_KEY`	Clé API Anthropic	✅ Oui
`FASTDOC_OUTPUT_DIR`	Dossier de sortie (override config)	Non
`FASTDOC_LANG`	Langue de la documentation (`fr`, `en`)	Non

5. GitHub Action

5.1 Exemple d'utilisation

```yaml
# .github/workflows/docs.yml
name: Generate Documentation

on:
  push:
    branches: [main]

jobs:
  docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: your-username/fastdoc@v1
        with:
          api-key: ${{ secrets.ANTHROPIC_API_KEY }}
          output-format: md
          output-dir: ./docs
```

5.2 Inputs de l'action

Input	Description	Défaut
`api-key`	Clé API Anthropic (secret)	—
`output-format`	Format de sortie (`md`, `json`, `all`)	`md`
`output-dir`	Dossier de sortie	`./docs`
`language`	Langue de la documentation	`en`
`commit-docs`	Committer automatiquement les docs	`true`

6. Critères d'acceptance (Definition of Done)

6.1 Fonctionnel

- [ ] La commande `fastdoc generate` fonctionne sur un projet JS/TS sans configuration
- [ ] La commande `fastdoc generate` fonctionne sur un projet PHP/Laravel
- [ ] La commande `fastdoc generate` fonctionne sur un projet Dart/Flutter
- [ ] La commande `fastdoc generate` fonctionne sur un projet Python
- [ ] La documentation générée contient toutes les fonctions/classes du fichier analysé
- [ ] Les fichiers Markdown et JSON sont générés correctement
- [ ] La commande `fastdoc init` crée un fichier `.fastdocrc` valide

6.2 Technique

- [ ] Couverture de tests >= 70 %
- [ ] Temps de génération < 30s pour un fichier de 500 lignes
- [ ] Gestion des erreurs (API indisponible, fichier non supporté, clé manquante)
- [ ] Le package est publié sur npm et installable via `npm install -g fastdoc`
- [ ] La GitHub Action fonctionne sur un repo de test

6.3 Qualité

- [ ] Code 100 % TypeScript strict
- [ ] ESLint sans erreurs
- [ ] README complet avec exemples

---

7. Roadmap

v1.0 — MVP

- Support JS/TS, PHP, Dart, Python, Node.js, NestJS
- CLI avec commande `generate` et `init`
- Output Markdown + JSON
- GitHub Action basique

v1.1

- Support de fichiers de configuration avancés
- Mode `--watch` (re-génération à chaque modification)
- Support Java / Kotlin

v2.0

- Interface web pour visualiser la documentation
- Génération de diagrammes UML (classes, séquences)
- Intégration Notion / Confluence

---

8. Contraintes et risques

Contrainte / Risque	Impact	Mitigation
Coût API Anthropic	Moyen	Mise en cache des résultats, limite de tokens configurable
Qualité variable de la doc générée	Élevé	Prompt engineering soigné, tests sur fixtures réelles
Taille des fichiers volumineux	Moyen	Découpage en chunks, analyse par fonction
Support de tous les langages	Moyen	Lancement avec les langages maîtrisés, extension progressive

---

Document rédigé le 12 mars 2026 — Version 1.0